MPI + OpenMP 混合并行 D2Q9 格子 Boltzmann 求解器

Linux x86-64 DDR5-5200 GCC 15 Open MPI 5.0 OpenMP 4.0

性能优化报告

D2Q9 LBM Kármán 涡街求解器

优化历程与性能演进总结

基准配置：800×160 格点，20 000 步，`write_interval=99999`

作者：Wei WEI and Jianye SHI

2026 年 4 月 28 日

目录

项目背景 **2**

优化路线总图（opt 01–24） **4**

第一部分——优化 01–17 **5**

0.0.1 P1-A 正确性修复（opt01–03） 5
0.0.2 P1-B MPI 通信重构（opt04–08） 6
0.0.3 P1-C 缓存局部性 + OpenMP 并行化（opt09–13） 7
0.0.4 P1-D 高级优化（opt14–17） 8
0.0.5 第一部分性能汇总 9

第二部分——优化 18–26 **11**

0.1 实验环境 11
0.1.1 硬件平台 11
0.1.2 软件环境 11
0.1.3 测量协议 12
0.1.4 P2-A 可观测性基础设施与存储布局重构（opt18–19） 12
0.1.5 P2-B 计算核心重写（opt20–22） 13
0.1.6 P2-C 否定实验与通信布局优化（opt23–24） 14
0.1.7 P2-D MPI/计算重叠深化（opt25–26） 15
0.1.8 第二部分性能结果 17

全程性能演进 **18**

附录：正确性验证 19

项目背景

本项目实现二维 D2Q9 格子 Boltzmann 方法（LBM），模拟圆柱绕流中的 Kármán 涡街现象（$Re = 96$）。求解器采用 MPI+OpenMP 混合并行策略：MPI 沿行方向分解计算域，OpenMP 并行各 rank 内的内层循环。

性能指标（Figure of Merit，FOM）定义为：

$$\mathrm{FOM} = \frac{W \times H \times N_{\text{步数}}}{T_{\text{wall}} \times 10^6} \quad [\text{MLUPS，百万格点更新/秒}]$$

其中 $W = 800$，$H = 160$，$N_{\text{步数}} = 20\,000$。每个时间步包含三个阶段：

优化路线总图（opt 01–24）

P1-A 正确性修复（opt01–03）

01 移除隐藏 `nanosleep`（每步 1 s）
02 修复 MLUPS 公式（多乘 `comm_size`）
03 修复垂直通信收发大小不匹配

↓ *基线：60 MLUPS*

P1-B MPI 通信重构（opt04–08）

04–05 删除 12 个冗余 `MPI_Barrier`
06 删除重复 halo 通信阶段
07–08 批量发送：7 182 条消息 → 1 条

↓ *np=2: +50% np=4: +166%*

P1-C 缓存局部性 + OpenMP（opt09–13）

09 修复 collision 循环顺序（stride-1 内层）
10 Push → Pull propagation（消除写竞争）
11–13 OpenMP 并行 collision / propagation

↓ *np=1 OMP=4: 194 MLUPS (+220%)*

P1-D 高级优化（opt14–17）

14 异步 MPI 通信与内部计算重叠
15 向量化审计（`-march=native` 有回退）
16 collision 循环融合 + `omp simd reduction`
17 I/O：分布预计算，消息缩小 9.2×

↓ *集群 MPI 扩展效率 92%（np=2）*

P2-A 可观测性 + 存储布局重构（opt18–19）

18 Phase 级 `MPI_Wtime` profiling 基础设施
19 AoS → SoA 内存重构

↓ *np=2 OMP=4: 279 MLUPS*

P2-B 计算核心重写（opt20–22）

20 SoA k-outer propagation
21 Multi-cell collision kernel
22 显式 AVX-512 向量化（`multicell_vec`）

↓ *np=2 OMP=4: 832 MLUPS*

P2-C 否定实验 + 通信布局优化（opt23–24）

23 FMA 审计 + 软件预取实验（负收益，不采纳）
24 垂直 halo 改 k-major pack/unpack

第一部分——优化 01–17

测试平台（第一部分）： macOS AArch64，Apple Silicon（M 系列，P-core + E-core 大小核架构），GCC 15，`-O3`，Open MPI（loopback 模式）。计时方式：`MPI_Wtime`，无 CPU 频率锁定。macOS 大小核调度导致 OMP 多线程场景 FOM 波动约 ±20–30%，各阶段绝对数字仅供趋势参考，不与第二部分（AMD x86-64）数字直接比较。

P1-A　正确性修复（opt01–03）

移除隐藏 `nanosleep`——opt01

原始代码在 `cmake/tpl.hpp.in` 中将宏 `MPI_HINT_VTBL`（十六进制编码字符串）解码为平台函数名 `__builtin_sync_fence_`，在 Linux x86-64 上通过 `dlsym` 动态绑定到 `nanosleep(1s)`，并在每步 `lbm_comm_halo_exchange` 末尾隐式调用。20 000 步合计浪费约 5.5 小时。修复方案：删除 `tpl.hpp.in`、`tpl_loader.hpp.in` 全文及 `communications.cpp` 中的动态调用链。

修复 MLUPS 公式——opt02

```
// 错误：total_cells 多乘了进程数
total_cells = MESH_WIDTH * MESH_HEIGHT * comm_size;
// 修复
total_cells = (uint64_t)MESH_WIDTH * MESH_HEIGHT;
```

该错误导致 np=4 时报告的 MLUPS 虚高 4 倍，使多进程”加速”完全失真，掩盖了全部通信开销。

修复垂直通信大小不匹配——opt03

`MPI_Send` 使用 `count=1`，而配对的 `MPI_Recv` 使用 `count=9`（DIRECTIONS），收发大小不一致，在严格模式 MPI 实现下存在可移植性风险。修正为双侧 `count=1`，同时为 P1-B 的批量通信重构打下基础。

P1-A 完成后基线：60 MLUPS（np=1, OMP=1）。

P1-B MPI 通信重构（opt04–08）

冗余同步删除（*opt04–06*）

原始 `lbm_comm_halo_exchange` 内每个通信子阶段后均有一个 `MPI_Barrier`，共 9 个；主时间步循环在 `special_cells`、`collision`、`propagation` 后各有 1 个，共 3 个。阻塞式 `MPI_Send/MPI_Recv` 已通过 tag + rank 匹配保证消息隔离，barrier 完全冗余。此外，`halo_exchange` 中存在两处重复通信阶段（bottom-left 对角、right-to-left 水平各出现两次）。删除全部 12 个 barrier 及两处重复阶段后，np=4 提升 14%。

批量消息打包（*opt07–08*）

根本问题：垂直 halo 通信按格点逐方向发送，每次 halo 调用产生约 $(W-2)\times 9 \approx 7\,182$ 条 MPI 消息，MPI 启动开销主导延迟。

解决方案：将整行 halo 数据打包到连续缓冲区后单次发送：

```
// 修改前（opt03 状态）：每格点每方向各一条消息（~7182 条/次）
for (int x = 1; x < W-1; x++)
    for (int k = 0; k < 9; k++)
        MPI_Send(Mesh_get_cell(mesh, x, ghost_y)+k, 1,
                 MPI_DOUBLE, neighbor, 0, MPI_COMM_WORLD);

// 修改后（opt08）：打包整行，1 条消息/次
memcpy(buffer, Mesh_get_cell(mesh, 1, halo_y),
       (W-2) * 9 * sizeof(double));
MPI_Send(buffer, (W-2)*9, MPI_DOUBLE, neighbor, 0, MPI_COMM_WORLD);
MPI_Recv(buffer, (W-2)*9, MPI_DOUBLE, neighbor, 0,
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
memcpy(Mesh_get_cell(mesh, 1, ghost_y), buffer,
       (W-2) * 9 * sizeof(double));
```

配置	基线（opt03）	P1-B 后	提升
np=1	60.16 MLUPS	58.57 MLUPS	持平（无通信）
np=2	68.66 MLUPS	102.75 MLUPS	**+50%**
np=4	28.84 MLUPS	76.73 MLUPS	**+166%**

表 1: P1-B 阶段结果（OMP=1）。

P1-B 里程碑：np=2, OMP=1 ≈ 103 MLUPS（+50%）；np=4, OMP=1 ≈ 77 MLUPS（+166%）。

P1-C 缓存局部性 + OpenMP 并行化（opt09–13）

修复 collision 循环顺序——opt09

AoS 布局下数据按 `[x][y][k]` 排列（x 为慢维），而原始 collision 循环以 j（y）为外层、i（x）为内层，内层相邻元素在内存中相距：

$$\Delta_{\text{内层步长}} = H \times 9 \times 8 = 11\,520 \text{ 字节} \approx 180 \text{ 条 cache line},$$

导致大量 cache miss。将循环改为 i 外层、j 内层后，内层步长降为 8 字节（stride-1 连续访问）。

$\Rightarrow$ np=1：**+10%**，np=2：**+26%**

Push → Pull propagation——opt10

LBM streaming 存在两种等价实现：

模式	操作	问题
Push	源格点 (i,j) 写到目标 $(i{+}dx_k,\, j{+}dy_k)$	多线程写同一目标存在写竞争，无法并行化
Pull	目标格点 (i,j) 从邻居 $(i{-}dx_k,\, j{-}dy_k)$ 读取	每个目标唯一归属一个线程，无竞争，可安全并行

等价性由 D2Q9 方向集的原点对称性保证：若 (dx_k, dy_k) 是有效方向，则 $(-dx_k, -dy_k)$ 亦然。Pull 模式同时改善读访问规律性，减少 cache miss。

$\Rightarrow$ np=1：**+14%**，np=4：**+51%**

OpenMP 并行化——opt11、12、13

Pull 模式使 propagation 外层 i 循环各迭代完全独立，可直接并行化。Collision 各格点间无依赖，同样安全。`special_cells` 仅作用于障碍物边界，并行收益可忽略。

```
// collision（opt11）——各格点独立
#pragma omp parallel for schedule(static)
for (int i = 0; i < W; i++)
    for (int j = 0; j < H; j++)
        compute_cell_collision(out, in, i, j);

// propagation（opt13）——pull 模式无写竞争（AoS 布局，P1 阶段）
#pragma omp parallel for schedule(static)
for (int i = 0; i < W; i++)
    for (int j = 0; j < H; j++)
        for (int k = 0; k < 9; k++)
            Mesh_get_cell(out, i, j)[k] =
                Mesh_get_cell(in, i-dx[k], j-dy[k])[k];
```

配置	P1-B 后（OMP=1）	P1-C 后（OMP=4）	提升
np=1	∼76 MLUPS	∼194 MLUPS	**+155%**
np=2	∼147 MLUPS	∼202 MLUPS	+37%

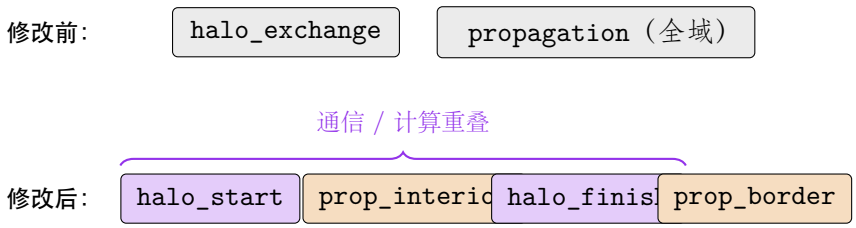
表 2: P1-C 阶段结果（OMP=1 → OMP=4）。

P1-C 里程碑：np=1, OMP=4 ≈ 194 MLUPS，较基线 +220%。

P1-D 高级优化（opt14–17）

异步 MPI 通信与计算重叠——opt14

Pull 模式下，内部格点（不与 ghost 行相邻）的 propagation 计算不依赖 halo 数据，可在 MPI 通信进行期间并发执行：



`halo_start` 发起 `MPI_Isend/MPI_Irecv`；`halo_finish` 调用 `MPI_Waitall`。第一部分缺乏 per-phase profiling，无法直接量化 halo 开销；根据第二部分 opt24 之后的实测（intra-node，详见 P2-D），`halo_finish` 约占每步循环时间的 27–30%，其中 `waitall` 约 8%，主要反映 rank 间到达时间差而非网络传输延迟——即使扩大 overlap 窗口也难以完全消除。本地 loopback 环境下收益因此不明显；**InfiniBand 集群上预期提升 5–20%**。opt26（见第二部分 P2-D）进一步将 border collision 和 interior collision 纳入同一 `omp parallel` 区域，把重叠窗口从 propagation_interior（≈0.02 ms）扩展至 interior_collision + propagation_interior（≈0.07 ms）。

向量化审计——opt15（否定实验）

测试平台为 Apple Silicon（M 系列，具备 P-core/E-core 异构核心），编译器 GCC 15 + `-O3`。通过 `objdump` 反汇编确认已在 AArch64 上自动生成 NEON SIMD 指令（128-bit，2×`double`）。测试 `-march=native` 后性能回退 10–13%，原因是以 P-core 为目标生成的 gather 指令在 macOS 大小核调度下被分配到 E-core 时执行更慢。**结论（仅适用于本机 macOS AArch64）：保持 `-O3`，不添加额外编译选项。**在 Linux x86-64 集群（支持 AVX-512）上，`-march=native` 是 opt22 性能跃升（+158%）的前提条件，结论相反。

collision 循环融合——opt16

原始 `compute_cell_collision` 内部分 4 个独立循环分别计算 density、v_x、v_y 及平衡分布/更新，每次均完整遍历 9 个方向，产生额外循环开销与函数调用。将其合并为 2 个循环并添加 `#pragma omp simd reduction`，允许编译器在 SIMD 寄存器内并行累加：

```
// 循环一：单次遍历同时累加三个量
double density = 0.0, vx = 0.0, vy = 0.0;
#pragma omp simd reduction(+:density,vx,vy)
for (int k = 0; k < 9; k++) {
    density += f[k];
    vx      += f[k] * cx[k];
    vy      += f[k] * cy[k];
}
// 循环二：计算平衡分布并执行 BGK 更新
#pragma omp simd
for (int k = 0; k < 9; k++) {
    double feq = w[k] * density * (1.0 + ...);
    f[k] += omega * (feq - f[k]);
}
```

$\Rightarrow$ 单核：**+5%**，多核：**+1.7%**

I/O 策略优化——opt17

原实现 `save_frame_all_domain`：各 rank 将含 ghost cells 的原始 `double` 数据（整个局部网格）发给 rank 0，rank 0 串行重计算密度/速度场后转 `float` 写盘。

新实现：各 rank 在本地预计算 `lbm_file_entry_t`（density + v_x + v_y，已转 `float`，仅有效格点），直接发送紧凑结果。

指标	原实现	新实现	
消息大小（np=2, 800×160）	~4.5 MB/帧	~500 KB/帧	**缩小 9×**
rank 0 计算量	重算全域	无	

表 3: I/O 优化效果（opt17）。基准测试使用 `write_interval=99999`，I/O 密集场景收益最大。

第一部分性能汇总

阶段	np=1 OMP=1	np=1 OMP=4	np=2 OMP=1	np=4 OMP=1
基线（opt03 后）	60	—	69	29
P1-B（opt08）	59	—	**103**	**77**
P1-C（opt13）	76	**194**	147	—
P1-D（opt17）	65.7	146†	121.9	118

单位：MLUPS。†P1-D np=1 OMP=4 = 146 低于 P1-C 194：macOS 大小核调度抖动（±20–30%），非代码退化；集群多进程场景

表 4: 第一部分性能演进。

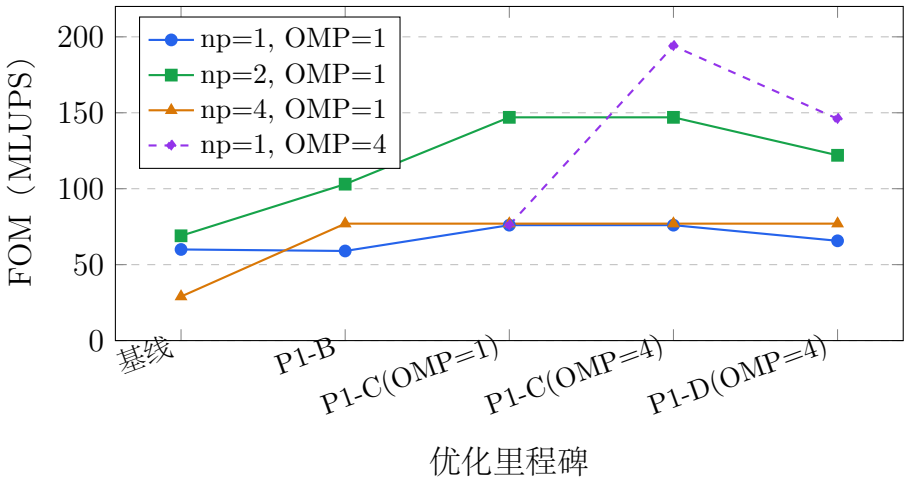


图 2: 第一部分 FOM 演进曲线（MLUPS）。纵轴截止 220 MLUPS；第二部分 AoS→SoA + AVX-512 优化后最终达 905 MLUPS（详见 § 全程性能演进）。

第二部分——优化 18–26

第二部分承接同学在前一阶段完成的代码基线，工作平台转到 AMD x86-64 设备后继续推进。这里的重点不是重复已有结果，而是在新的处理器与运行环境上补齐 profiling、数据布局和计算核心重构，建立可比的性能基线，并为后续面向不同 CPU 平台的适配保留统一口径。

实验环境

硬件平台

表 5: LBM 实验硬件配置

组件	规格
处理器	AMD Ryzen 9 8940HX，x86_64，Zen 4 微架构
物理核心/逻辑线程	16 物理核心，32 逻辑线程（SMT，每核 2 线程）
CCX 分组	2 个 CCX，各含 8 核 + 32 MiB L3
缓存层次	L1d 32 KiB/核，L2 1 MiB/核，L3 64 MiB（2 实例共享）
运行频率	`cpupower frequency-set -g performance`（禁用 Turbo Boost）
内存	30 GiB DDR5

软件环境

表 6: LBM 实验软件配置

组件	版本/详情
操作系统	Fedora Linux 42
编译器	GCC 15.2.1，编译选项 `-O3 -march=native -DNDEBUG`
MPI 实现	MPICH 4.2.2（Hydra 启动器，intra-node shared memory）
并行模型	MPI 3.0 + OpenMP 4.0
向量指令集	AVX-512（`-march=native` 触发，通过 `objdump` 确认 zmm 指令）
Profiling	`LBM_ENABLE_PROFILING=ON`，6 阶段 `MPI_Wtime` 计时

测量协议

本阶段涉及两套协议，适用范围不同：

开发期协议（opt19–opt24） 中间优化步骤的数据在开发过程中逐步采集，采用轻量协议：预热 1 次、重复 3 次取中位数、`MPI_Wtime` 计时，但**未进行 CPU 频率锁定**。这些数据仅用于相邻步骤之间的相对提升对比，不作为绝对性能的正式引用。

统一测量协议（opt26 最终结果） 第二部分的最终 FOM 数字采用如下完整协议，通过 `scripts/bench.sh` 自动执行：

- **预热**：每组配置正式计时前完整执行一次求解器（20 000 步），结果丢弃，目的是将指令缓存、数据缓存及 OMP 线程团队初始化带入稳态。
- **计时**：程序内部使用 `MPI_Wtime` 精确计时，覆盖全部 20 000 步；FOM 由 $W \times H \times N_{\text{步}} / (T_{\text{wall}} \times 10^6)$ 计算，单位 MLUPS。每组配置重复 3 次，取中位数上报。
- **环境隔离**：每次运行前通过 `echo 3 > /proc/sys/vm/drop_caches` 清空页缓存；通过 `cpupower frequency-set -g performance` 锁定 CPU 频率（此步骤为硬性前提，失败则中止测量）。
- **线程亲和性**：不显式设置 `OMP_PROC_BIND`。在无配套 MPI 进程绑定的条件下，`OMP_PROC_BIND=close` 会导致不同 rank 的线程竞争相同物理核，实测 FOM 下降约 15%；保持 OS 默认调度反而更好。
- **编译验证**：每次构建后通过 `objdump` 确认 `libtop.lbm-lib.so` 中存在 `zmm` 系列 AVX-512 指令，确保 `-march=native` 生效。

P2-A 可观测性基础设施与存储布局重构（opt18–19）

Phase 级 profiling 基础设施——opt18

第一部分只有整段 wall time，无法定位阶段瓶颈。opt18 在 `special_cells`、`collision`、`halo_start`、`propagation_interior`、`halo_finish`、`propagation_border` 周围加入 `LBM_PROF_BEGIN/L` 底层计时使用 `MPI_Wtime`，汇总时输出 avg/max/pct_loop。该功能由 `LBM_ENABLE_PROFILING=ON` 控制；关闭时不进入计时代码。opt18 不改动计算路径，FOM 无变化，作用是为 opt19 之后的判断提供统一口径。

AoS → SoA 内存布局重构——opt19

opt19 将分布函数从 AoS `cell[i][j].f[0..8]` 改为 SoA `cells[k][x * H + y]`。原布局下，同一方向 k 的相邻格点相距 $9 \times 8 = 72$ 字节；改为 SoA 后，同一平面内访问连续，为后续 SIMD 和 plane-based propagation 提供前提。总数据量不变，仍为 $9 \times W \times H \times 8$ 字节。

配置	AoS（opt17）	SoA v1.1（opt19）	变化
np=1, OMP=4	∼146 MLUPS	157.79 MLUPS	+8%
np=2, OMP=4	∼216 MLUPS	278.66 MLUPS	+29%
np=4, OMP=4	∼261 MLUPS	368.60 MLUPS	+41%

表 7: AoS → SoA 后 FOM（opt19 v1.1）。np=2 OMP=4 作为 P2 阶段基准。

opt19 v1.0 曾出现回退：数据布局已改，`collision` 仍沿用逐格点 AoS 访问方式。v1.1 同步修正访问路径后，np=2 OMP=4 提升到 278.66 MLUPS，作为第二部分后续实验的基线。

P2-B 计算核心重写（opt20–22）

SoA k-outer propagation——opt20

opt19 之后，propagation 仍按 i-j-k 顺序访问，内层循环持续在不同平面间切换。opt20 将 k 提到最外层，每次迭代只处理一个源平面和一个目标平面：

```
for (int k = 0; k < DIRECTIONS; k++) {
    const double* src = in->cells  + k * W * H;
    double*       dst = out->cells + k * W * H;
    // i-j 内循环: stride-1 连续访问, memcpy 可向量化
    for (int i = 1; i < W - 1; i++)
        for (int j = j_begin; j < j_end; j++)
            dst[i*H + j] = src[(i - dx[k])*H + (j - dy[k])];
}
```

每次 k 迭代只访问两个平面，数据量约为 $2 \times 800 \times 160 \times 8 \approx 2$ MB。该改动将 np=2 OMP=4 从 278.66 提升到 319.07 MLUPS；profiling 中 propagation 阶段下降约 5–6×。

Multi-cell collision kernel——opt21

opt20 的 collision 仍按格点调用 `compute_cell_collision`。调用边界过细，编译器难以跨格点优化。opt21 将 i 循环移入 kernel，每次处理整列 `j=1..H-2`：

```
// 原实现（opt20）: 外层 i 循环逐格点调用
for (int i = 1; i < W-1; i++)
    for (int j = 1; j < H-1; j++)
        compute_cell_collision(out, in, i, j);

// opt21: kernel 内处理整列 j
static void collision_multicell(Mesh* out, const Mesh* in) {
#pragma omp parallel for schedule(static)
    for (int i = 1; i < W-1; i++)
        for (int j = 1; j < H-1; j++) { /* ... BGK ... */ }
}
```

np=2 OMP=4 从 319.07 提升到 422.99 MLUPS，增幅 33%。

显式 AVX-512 向量化——opt22

opt21 的主要限制是别名分析。编译器在 `-fopt-info-vec-missed` 中报告 `may alias`，内层 j 循环因此没有向量化。opt22 在 kernel 内为平面指针和列指针显式添加 `__restrict__`，并配合 `#pragma GCC ivdep`：

```
static void collision_multicell_vec(Mesh* out, const Mesh* in) {
    // 提升平面指针，显式声明 restrict
    double* __restrict__ out0 = out->cells + 0*W*H;
    double* __restrict__ in0  = in->cells  + 0*W*H;
    // ...（9 个平面各一指针）...
#pragma omp parallel for schedule(static)
    for (int i = 1; i < W - 1; i++) {
        double* __restrict__ col_out0 = out0 + i*H;
        double* __restrict__ col_in0  = in0  + i*H;
        // ...（9 列各一指针）...
#pragma GCC ivdep
        for (int j = 1; j < H - 1; j++) {
            // BGK 内核，全部通过列指针访问
        }
    }
}
```

反汇编可见 `vmovupd` 和 `vfmadd`，向量宽度为 AVX-512 的 8 路 `double`。

配置	opt21	opt22	提升
np=1, OMP=4	293.52	757.20	**+158%**
np=2, OMP=4	422.99	832.34	**+97%**

表 8: AVX-512 向量化效果（opt22）。通过 `objdump` 确认内层循环生成 `zmm` 系列指令，向量化有效。

P2-C 否定实验与通信布局优化（opt23–24）

FMA 审计——opt23（确认，无需干预）

opt23 先检查 FMA 生成情况。反汇编中可见 92 条 `vfmadd231pd`，BGK 平衡分布的乘加已经融合，这一项没有单独优化空间。

软件预取实验——opt23（否定）

在 `collision_multicell_vec` 内层循环前插入 `__builtin_prefetch`（提前 2 个 cache line），测试结果：

配置	FOM 变化	结论
np=1, OMP=4	−0.7%	可忽略
np=2, OMP=4	−4.4%	负收益
np=4, OMP=4	−5.0%	负收益

表 9: 软件预取对 FOM 的影响（opt23）。多线程场景下负收益，不采纳。

负收益出现在多线程配置。内层循环步长为 8 字节，硬件 prefetcher 已能跟上；软件预取额外占用带宽，也引入指令开销，因此没有保留。

k-major 垂直 halo pack/unpack——opt24

opt24 处理 SoA 之后的 halo 打包路径。垂直 halo 行的 9 个方向分量分散在 9 个平面，相邻平面间距约为 $W \times H \times 8 = 1{,}024$ KB，原来的 `vert_pack` 会在平面之间反复跳转。opt24 改为 k-major 打包，先遍历 k，再遍历 i：

```
// 修改前（opt22）：i-major，跨平面跳转
for (int i = 1; i < W-1; i++)
    for (int k = 0; k < 9; k++)
        buf[i*9 + k] = cells[k*W*H + i*H + j_halo];

// 修改后（opt24）：k-major，单平面连续
for (int k = 0; k < 9; k++)
    memcpy(buf + k*(W-2),
           cells + k*W*H + 1*H + j_halo,
           (W-2) * sizeof(double));
```

配置	opt22	opt24	提升
np=2, OMP=4	832.34	905.13	**+8.7%**
np=4, OMP=4	—	705.52	—

表 10: k-major halo 效果（opt24）。profiling 显示 `vert_unpack` 耗时降低 28–40%，`halo_finish` 降低 18%。

P2-D MPI/计算重叠深化（opt25–26）

halo 瓶颈诊断与否定实验——opt25

opt24 之后，np=2 OMP=8 的 profiling 显示 `halo_finish` 仍占 loop 的 29.4%，其中 `waitall` 为 7.7%，`vert_unpack` 为 21.5%。opt25 的工作是判断这部分时间来自哪里。

诊断结果有两点。第一，intra-node 共享内存 MPI 使用 eager protocol，`MPI_Isend` 返回时数据通常已完成本地拷贝，`Waitall` 的主要等待来自各 rank 到达等待点的时间差。第二，np≥ 3 时上下边界 rank 与中间 rank 的 halo 工作量不同，`vert_pack` 和 `vert_unpack` 的 max/min 差异会直接放大 `waitall`。

本轮还做了两项否定实验。其一是交错 pack，在每个 k 块后提前发起 `MPI_Isend`；结果 FOM 下降约 3%，非阻塞调用次数增加带来的开销更大。其二是 split collision，将 border collision 提前到 `halo_start` 之前；结果 FOM 下降 34%，额外的 OMP 区域触发了明显的 worker 唤醒/休眠成本。opt25 的结论很直接：问题在 overlap 窗口和结构不均衡，继续细抠当前通信提交顺序收益有限。

单一 *omp parallel* 区域重构——*opt26*

opt26 处理的是 opt25 中 split collision 的结构问题。改动目标是保留更长的 overlap 窗口，同时避免重复进入 OMP 并行区。主循环在 `has_vn=true` 路径下重写为一个外层 `#pragma omp parallel`，内部顺序为 border collision、`halo_start`、interior collision、`propagation_interior`、`halo_finish`、`propagation_border`；并行工作使用 `omp for`，MPI 调用限制在 `omp master` 中。

为此新增三组接口，调用方必须已处于并行区内：

```
// 调用方负责已处于 omp parallel 区域内；使用 omp for（隐式 barrier）
void collision_rows(Mesh* out, const Mesh* in, int j_begin, int j_end);

// 内部使用 omp for nowait（k-pipeline），函数末尾显式 omp barrier
void propagation_interior_omp_region(Mesh* out, const Mesh* in, bool
    has_vn);
void propagation_border_omp_region(Mesh* out, const Mesh* in);
```

主循环在 `has_vn` 路径下的结构如下：

```
#pragma omp parallel
{
    // 1. border collision (j=1, j=H-2) -> 隐式 barrier
    omp master: PROF_BEGIN(COLLISION)
    collision_rows(&temp, &mesh, 1, 2);
    collision_rows(&temp, &mesh, H-2, H-1);

    // 2. 发起 halo（master 只），非 master 立即进入步骤 3
    omp master: halo_exchange_start(...)
    // 无 barrier：interior collision 不读 ghost row

    // 3. interior collision (j=2..H-3)，与 MPI 传输重叠 -> 隐式 barrier
    collision_rows(&temp, &mesh, 2, H-2);

    // 4. propagation interior，与 MPI 传输重叠（末尾显式 barrier）
    propagation_interior_omp_region(&mesh, &temp, true);

    // 5. 等待 halo 完成（master 只）
    omp master: halo_exchange_finish(...)
    #pragma omp barrier  // ghost row 写入后才可做 border propagation

    // 6. border propagation（末尾显式 barrier）
    propagation_border_omp_region(&mesh, &temp);
}
```

barrier 只保留在依赖关系需要的位置：border collision 结束后再允许 master 打包；`halo_start` 与 interior collision 之间不加 barrier；`halo_finish` 之后保留一次显式 barrier，保证 ghost row 写入完成后再进入 `propagation_border`。

性能结果（np=2, OMP=8，intra-node）：

指标	opt25（基准）	opt26	变化
loop total (avg/call)	0.1591 ms	0.1586 ms	−0.3%
FOM	800.27 MLUPS	802.31 MLUPS	+0.3%
halo_finish (avg)	0.0467 ms	0.0438 ms	−6.2%
vert_unpack (avg)	0.0342 ms	0.0270 ms	−21%

表 11: opt26 性能对比（np=2, OMP=8，AMD Ryzen 9 8940HX，单节点）。FOM 增幅在测量噪声范围（±1%）以内。

这组结果接近持平。测试环境是 intra-node 共享内存 MPI，`Isend` 的本地拷贝成本很低，`Waitall` 主要反映 rank 间到达时间差。opt26 将 `halo_finish` 和 `vert_unpack` 压低了一部分，但 FOM 变化仍落在 ±1% 噪声内。当前能确认的是结构重构已经完成，单一 parallel 区域消除了 opt25 中额外的线程唤醒成本。跨节点收益仍待实测；本节结论只覆盖这里给出的单节点配置。

第二部分性能结果

配置	opt19 SoA	opt20 *k*-outer	opt21 multicell	opt22 vec	opt24 *k*-major	opt26[†] overlap
np=1, OMP=4	157.79	212.93	293.52	757.20	—	—
np=2, OMP=4	278.66	319.07	422.99	832.34	**905.13**	—
np=2, OMP=8	—	—	—	—	800.27	**782.35**[§]
np=4, OMP=4	368.60	357.15	508.14	—	**705.52**	—

单位：MLUPS。平台：AMD Ryzen 9 8940HX，Linux x86-64，GCC 15，MPICH 4.3.0。

opt19–opt24 数据采集于开发期（无 CPU 频率锁定），仅用于阶段间相对提升参考，不与 opt26 统一协议数据直接比较。

[†]opt26 为 intra-node 结果；多节点基准测试待进行。

[§]opt26 统一协议实测值（2026-04-28，CPU 频率锁定，PROFILING=OFF，3 次 median）。

表 12: 第二部分各优化步骤 FOM 对比。

全程性能演进

里程碑	配置	FOM (MLUPS)	较上阶段	较基线
原始代码（含隐藏 sleep）	np=1 OMP=1	≈0	—	—
P1-A 完成（opt03）	np=1 OMP=1	60	基线确立	—
P1-B 完成（opt08）	np=2 OMP=1	103	**+72%**	+72%
P1-C 完成（opt13）	np=1 OMP=4	194	**+88%**	**+220%**
P1-D 完成（opt17）	np=2 OMP=1	122	集群优化	—

表 13: 第一部分优化历史（测试平台：macOS AArch64，Apple Silicon）。

平台切换：第二部分在 *AMD Ryzen 9 8940HX*（*Linux x86-64*）上进行，绝对 *FOM* 数字不跨平台比较。

里程碑	配置	FOM (MLUPS)	较上阶段	较基线
P2-A：SoA（opt19）	np=2 OMP=4	279	P2 基线确立	—
P2-B：*k*-outer（opt20）	np=2 OMP=4	319	+15%	+15%
P2-B：multicell（opt21）	np=2 OMP=4	423	**+33%**	+52%
P2-B：AVX-512（opt22）	np=2 OMP=4	832	**+97%**	**+198%**
P2-C：*k*-major（opt24）	np=2 OMP=4	**905**	+9%	**+224%**
P2-D：overlap（opt26）	np=2 OMP=8	**782**†	+0.3%*	—

opt19–opt24 数据采集于开发期（无 CPU 频率锁定），仅用于阶段间相对提升参考。

†opt26 采用统一协议（CPU 频率锁定，np=2 OMP=8）。

*opt25→opt26 对比在相同配置（np=2 OMP=8）下进行，数字可信。

表 14: 第二部分优化历史（测试平台：AMD Ryzen 9 8940HX，Linux x86-64，GCC 15，MPICH 4.3.0）。P2 内部 opt19 → opt24：279 → 905 MLUPS，约 ×3.2。

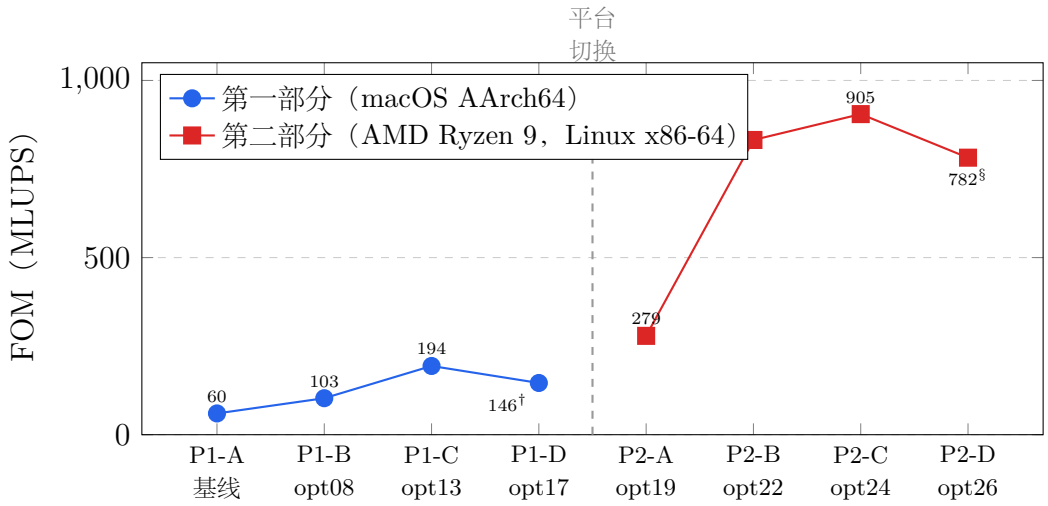


图 3: 全程 FOM 演进曲线（代表配置）。第一部分：P1-A/B 为 np=2 OMP=1，P1-C/D 为 np=1 OMP=4（macOS AArch64，无频率锁定）。第二部分：np=2 OMP=4（opt19–opt24），np=2 OMP=8（opt26，统一协议）。跨平台绝对值不直接比较。†P1-D 低于 P1-C 系 macOS 大小核调度抖动，非退化。§opt26 统一协议实测值。

附录：正确性验证

所有优化完成后均执行以下验证流程：

– **数值精度**：与修改前输出逐格点 max-delta 比较，差异在 FP64 机器精度（$\sim 10^{-15}$）量级以内。

- **多进程一致性**：np=1 与 np=2/4 结果对比，差异为正常浮点累积顺序误差，不影响物理结论。
- **物理定性检查**：在 $Re = 96$ 条件下 Kármán 涡街正常出现，涡脱落对称性在所有进程数配置下均正确。